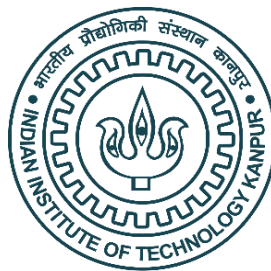


UDAAN TOOL - ANALYSIS

Machine Learning based **Post-Editing tool for Document** **Translation**



INDIAN INSTITUTE OF TECHNOLOGY KANPUR

KANPUR, 208016

MASTER OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

Under the Esteemed Supervision of Project Mentor

PROF. ARNAB BHATTACHARYA

Project Report for CS689(Computational Linguistics for Indian Languages)

Submitted By:

- DEEPAK PATNAIK BALARAMAMAHANTHI (231110010)
- NITISH KUMAR (231110033)
- SHIVAM MISHRA (231110047)

DECLARATION

We hereby declare that this report being submitted to the fulfilment of the 2nd semester project report submission of the subject: Computational Linguistics for Indian Languages (CS689), is carried out under Supervision of **Prof. Arnab Bhattacharya**.

This report presents entirely original work. We would also like to ensure that all the analysis and evaluations conducted in this report were performed with integrity and in accordance with ethical standards. Any data, results, or conclusions presented are authentic and have not been falsified or manipulated.

Name	Roll No.	Department
Deepak Patnaik Balaramamahanthi	231110010	M.Tech (CSE)
Nitish Kumar	231110033	M.Tech (CSE)
Shivam Mishra	231110047	M.Tech (CSE)

ACKNOWLEDGEMENTS

We would like to express our sincere gratitude to everyone who has contributed to the successful completion of our group project on " UDAAN - Machine Learning based Post-Editing tool for Document Translation". Their support, guidance, and expertise have been invaluable throughout this endeavor.

We would also like to thank our supervisor, Prof. Arnab Bhattacharya sir for his constant guidance and support throughout the duration of this project. We deeply appreciate the assistance and support provided by individuals associated with Project UDAAN, particularly Ayush Maheshwari (PHD, IITB), Anuja Dumada (Project Manager of Project UDAAN), Rahul Kharat and Ajay Ravindran (MTECH CSE, IITB)

Above all, we express gratitude for our seamless collaboration and contributions, which culminated the timely and successful completion of the project.

ABSTRACT

Effective language translation is essential for bridging communication gaps in diverse societies like India. However, the effectiveness of existing translation tools may vary when dealing with multiple domain adaptation and complex syntactic structures. This study aims to analyze an advanced translation tool tailored to the needs of the Indian linguistic landscape.

By leveraging state-of-the-art machine learning techniques and linguistic insights, the proposed tool seeks to enhance translation accuracy and efficiency, thereby bridging communication gaps and fostering linguistic inclusivity in India. The data set used for the translation model was sourced from an open-source dataset called '**SAMANANTAR**' which contain the Largest Publicly Available Parallel Corpora Collection for 11 Indic Languages.

This project addresses the challenges of translating between Indic languages, characterized by complex structures and diverse vocabularies. The developed tool leverages novel domain adaptation and syntactic modeling to enhance accuracy, especially when handling specialized texts. This improved translation technology has the potential to expand communication and information access for millions of Indic language speakers, benefiting sectors like education, government services, healthcare, and cross-cultural exchange within India's diverse society.

TABLE OF CONTENT

DECLARATION	ii
ACKNOWLEDGEMENTS	iii
ABSTRACT	iv
1. CHAPTER 1: INTRODUCTION	7
1.1 Statement of Problem	8
1.2 Aim of Study	9
1.3 Objectives of the Study	9
1.4 Significance of the Project	9
1.5 Scope of the Study	9
2. CHAPTER 2: CONCEPTUAL REVIEW	10
2.1 Machine Translation (MT)	10
2.2 Translation Memory (TM)	10
2.3 Computer-Aided Translation (CAT)	10
2.4 Neural Machine Translation (NMT)	10
2.5 Post-Editing (PE)	10
2.6 Optical Character Recognition (OCR)	10
2.7 Domain Aware Dictionaries	11

3. CHAPTER 3: EXISTING SYSTEM	12
4. CHAPTER 4: PROPOSED SYSTEM	14
5. CHAPTER 5: METHODOLOGY	15
5.1 Lexically Constrained Translation	15
5.2 Cross Language Span Prediction	19
5.3 Lexicon Suggestions Using SPEAR Library	21
5.4 Tesseract4	23
6. CHAPTER 6: SYSTEM IMPLEMENTATION	24
6.1 Installation Requirements	24
6.2 Workflow Pipeline	24
7. CHAPTER 7: ANALYSIS	29
CONCLUSION	31
BIBLIOGRAPHY	32

CHAPTER 1

INTRODUCTION

India is a vast country where roughly 780 languages are spoken including Hindi, Marathi, Marathi, and so on. It becomes difficult for a person to convey their thoughts to others keeping the right context in mind. Translation plays a vital role in bridging gaps between languages and cultures. Accurate and precise translation is crucial for effective communication, knowledge sharing, and understanding across linguistic barriers. However, a poorly translated text can not only lose its original meaning but also convey an entirely different and unintended message.

In 2011, the Tamil translation of the Microsoft Office instructions contained a rather embarrassing error. The English phrase "Click or drag to create a rectangular selection area" was translated into Tamil as "Click or apply grief to create a rectangular selection area." The nonsensical translation arose due to the incorrect meaning assigned to the English word "drag." Such errors in translating software products and technical documentation can seriously hamper the user experience for non-English users across India's multilingual population.

Traditionally, human translators have relied on methods like literal word-for-word translation or extensive use of translation memories (databases of pre-translated sentence pairs). However, these approaches have their pitfalls. Literal translations often fail to convey the nuances and context of the source text accurately. Overreliance on translation memories leads to inconsistent quality and an overload of replicated phrasing across documents.

With recent advances in Machine Translation (MT), post-editing outputs from MT systems has become commonplace. While more efficient than starting from scratch, existing post-editing methods and tools still have limitations. Most tools cannot handle input formats including PDFs and images. They also lack smart features to assist and guide human translators effectively during the post-editing process.

This is where UDAAN (Machine Learning based Post-Editing tool for Document Translation) offers a new and innovative approach. Developed by researchers, UDAAN combines the power of Neural Machine Translation (NMT) with intelligent post-editing capabilities tailored for human translators.

Some key strengths of UDAAN include:

- Handling diverse input formats including PDFs and images via Optical Character Recognition (OCR)
- Producing domain-aware translations by ingesting domain-specific dictionaries
- Interactive suggestions and guidance based on source-target/target-target lexicons
- Visualization of source-target sentence alignments to reduce cognitive load
- Tracking of user edits to provide smart global replacement suggestions
- Support for rich text formatting and outputting to multiple file formats

Through its unique features, UDAAN aims to streamline and enhance the human post-editing process, leading to more accurate, consistent, and high-quality translations compared to previous methods.

1.1 Statement of Problem

Existing translation tools have several limitations when handling the complexities of Indic languages and real-world translation scenarios. Machine translation systems often lack accuracy with domain-specific vocabulary and complex grammatical structures. Traditional CAT tools may not effectively support long documents, various file formats like PDF or images, or offer robust features for post-editing Indic language translations. The UDAAN tool seeks to address these shortcomings.

Apart from that, current Neural based Machine Translations (NMT) translate using neural networks trained on large data corpus, to perform generic translations making it context unaware apart from lagging behind human standards.

1.2 Aim of Study

This analysis aims to evaluate the UDAAN tool's effectiveness in addressing the challenges of Indic language translation. The study will investigate how UDAAN's domain-aware translation, OCR integration, post-editing interface, and other features improve translation accuracy, efficiency, and user experience compared to traditional translation workflows and what improvements can be done.

1.3 Objectives of the Study

To accomplish the project's purpose, the following objectives have been established:

- i. Analyzing the Reduced Post-Editing Effort in Document Translation
- ii. Exploring the working of the Tool for Domain-Specific Translations
- iii. Gaining insights on Usability and Workflow of the Tool.

1.4 Significance of the Project

UDAAN is the only available translation tool which can:

- i. Works independently without internet, preserving input text as art.
- ii. Project Udaan aims to produce over 500 books in native languages for engineering courses, breaking the language barrier.
- iii. Translating, post editing, proof-reading the content in regional languages.
- iv. Perform context aware translation

1.5 Scope of the Study

- Improved Neural Machine Translation
 - a. High Quality Machine Translation Model
 - b. Domain-aware dictionaries for Translation Model
- Efficient Writing with AI
 - a. Grammarly for Indian Languages
 - b. Smart colour codes to identify potential errors

CHAPTER 2

CONCEPTUAL REVIEW

2.1 Machine Translation (MT):

Machine translation is the process of automatically translating text from one language to another using computer algorithms. It involves training models on large datasets of parallel texts in different languages to learn the mappings between languages.

2.2 Translation Memory (TM):

A translation memory is a database that stores pairs of source and target language segments, along with their translations, in a structured format. Translation memories are used in computer-assisted translation (CAT) tools to aid translators in their work.

2.3 Computer-aided Translation (CAT):

Computer-aided translation refers to the use of software tools and technology to assist human translators in the translation.

2.4 Neural Machine Translation (NMT):

Neural Machine Translation is an approach to machine translation that uses artificial neural networks to translate text from one language to another. It can produce more fluent and accurate translations, especially for languages with complex syntactic structures.

2.5 Post-editing (PE):

Post-editing is a process in the field of translation where a human linguist or translator reviews and revises machine-generated translations to ensure accuracy, fluency, and adherence to the desired translation. The fact is AI technology is not advanced enough to completely replace humans in the process all of the time, so post editing is used.

2.6 Optical Character Recognition (OCR):

OCR is a technology that enables the conversion of different types of documents, such as scanned paper documents, PDF files, or images captured by a digital camera, into editable and searchable data.

2.7 Domain Aware Dictionaries:

The heart of the tool is its ability to provide domain aware translation which is performed using Domain aware dictionaries. While talking to Ajay Ravindran, we found out that developers of IITB used Commission for Scientific and Technical Terminology to scrap the dictionaries for domain knowledge. Later an MOU was signed between IITB and CSTT on through UDAAN project.

CHAPTER 3

EXISTING SYSTEM

GOOGLE TRANSLATE

Google Translate is a widely used online translation service provided by Google. It allows users to translate text between various languages instantly. The service utilizes machine learning algorithms and vast amounts of data to provide translations that are generally accurate and understandable.

One of the major drawbacks of Google Translate is its inconsistency and occasional inaccuracies, especially with complex sentences, idiomatic expressions, and context-dependent phrases. It may struggle with nuances, cultural references, and subtleties that require human understanding. Additionally, Google Translator's translations can sometimes lack fluency and naturalness, leading to awkward or nonsensical output.

Moreover, privacy concerns have been raised regarding the data collected by Google Translate, as users' text inputs are processed and stored by Google to improve the service, raising potential issues around data security and confidentiality. Furthermore, Google Translate may not always support lesser-known languages or dialects, limiting its accessibility and usefulness for certain language communities.

COMPUTER AIDED TRANSLATION (CAT)

Contemporary CAT tools revolutionize the translation process by harnessing Translation Memory (TM), a repository storing previously translated segments. This functionality resembles a bilingual find-and-replace function, allowing translators to quickly retrieve and reuse translations for similar or identical segments. TMs are often augmented with domain-specific terminologies, akin to bilingual glossaries, which aid in maintaining consistency and accuracy across translations within specialized fields.

However, while TMs offer significant benefits in terms of efficiency and consistency, they also present drawbacks. One notable challenge is the tendency toward over-recycling, wherein translators excessively reuse pre-existing translations. This over-reliance can lead to a decline in translation quality over time, as nuances, context, and evolving language patterns may be overlooked in favor of convenience. Moreover, the

repetitive use of identical or similar translations can introduce inconsistencies, particularly in documents with varied contexts or styles.

Furthermore, despite their usefulness, current CAT tools have limitations in terms of file format compatibility. While they excel in handling plain text documents, they often struggle with more complex formats such as PDFs or images. This limitation restricts their applicability, as many translation projects involve materials in diverse formats beyond simple text. Without robust support for these file types, translators are forced to resort to manual methods or third-party tools, leading to inefficiencies and potential loss of data fidelity.

In summary, while CAT tools leveraging TM technology have transformed the translation landscape, they are not without challenges. Balancing the benefits of TM reuse with the risks of over-recycling requires careful attention from translators. Additionally, addressing the limitations in file format compatibility is crucial for enhancing the versatility and effectiveness of these tools in meeting the evolving needs of translation projects.

NEURAL MACHINE TRANSLATION (NMT)

Neural Machine Translation (NMT) is a cutting-edge approach employing neural networks trained on vast datasets to translate text from one language to another, operating at both word-to-word and sentence-to-sentence levels. Its strength lies in the ability to capture intricate linguistic patterns and context, yielding translations of remarkable quality. This prowess stems from the neural networks' learned weights, honed through exposure to extensive linguistic data.

However, despite its impressive capabilities, NMT isn't infallible. One notable limitation lies in its handling of specialized terminology crucial in domain-specific content. While NMT excels at general language translation, it may falter when encountering highly technical or domain-specific terms, leading to inaccuracies or mistranslations. Consequently, the quality of machine translation (MT) still lags behind that of human translation, particularly in contexts where precise terminology is paramount.

In essence, while NMT represents a significant leap forward in automated translation technology, there remains a gap between machine-generated translations and the nuanced, contextually accurate output produced by human translators, particularly in specialized domains.

CHAPTER 4

PROPOSED SYSTEM

The authors proposed UDAAN, an open-source post-editing translation tool that addresses a few issues of poor translation quality, by incorporating lexically constrained translation and an approach to align the words more accurately to increase the translation quality. They also developed a tool, where users can edit the machine translation outcome. This can reduce manual editing efforts to quickly produce publishable-standard documents in several Indic languages. They also used SPEAR library for efficient lexicon suggestions which are the major issues in considering the translation accu.

CHAPTER 5

METHODOLOGY

To achieve the UDAAN Tool's specification these are the methodologies implemented:

5.1 Lexically Constrained Translation: Dictionary Constrained Disambiguation for Improved NMT:

- The Neural machine translation (NMT) offers excellent translation quality, but it may lack accuracy with specialized terms crucial for domain-specific content. The ideal solution involves lexically constrained NMT, where the system uses domain-specific dictionaries for contextually appropriate translation choices. This approach is especially important in multilingual environments, promoting accessibility by ensuring domain-specific translations are accurate and useful.
- Lexically constrained NMT approaches can be broadly divided into two categories: hard and soft.
 - In the hard category, all constraints are guaranteed to appear in the output sentence.
 - In the soft category: The constraints are not forced into the predicted sentence but are used to guide the translation process.
- In UDAAN, they used a lexically constrained NMT system that disambiguates between multiple candidate translations derived from dictionaries, by augmenting training data with multiple dictionary candidates to actively encourage disambiguation during training by implicitly aligning multiple candidate constraints, the method simply is a copy-and-disambiguation method.
- During training, constraints are sampled from a generic source-target dictionary, where each source phrase may have multiple candidate translations. These constraints are appended to the source sentences in a parallel corpus. The model is trained in a soft manner, to ensure least disrupt the fluency of the translation (since Hard-alignment methods copy lexical constraints in the exact same form and might therefore result in ingestion of incorrect morphological forms)
- The constraints to be injected in UDAAN for translation could be derived either from
 - a. In-domain source-target dictionaries (then the context is considered by the model to perform active translation)
 - b. user provided source-target constraints during interactive machine translation.

Let each source sentence $X = (x_1, x_2, \dots, x_S)$ of length S be associated with a

constraint set C derived from a dictionary. Each source word can be mapped to multiple constraints in C . For example, $(x_i, C_i^1, \dots, C_i^k)$ denotes k possible dictionary translations associated with the word x_i . Let the corresponding translated sentence Y $(y_1, y_2, \dots, y_T)$ be of length T .

Given a triplet of source sentence, constraints and target sentence (X, C, Y) , their objective is to choose constraint words or phrases from C and potentially include the in the target sentence Y . The source sentence is appended with C constraints, where each inter-phrase constraint C_i is separated by a symbol '<sep>'. In turn, candidate translations C_i^j for this i^{th} intra-phrase constraint are separated by another symbol '<isep>'. $\hat{X} = [X, <sep>, C_1^1, <isep>, C_1^2, <isep>, C_{12}, \dots, C_n, <eos>]$ where $<eos>$ is the end of sentence token. Then, the model is trained by maximizing a log-likelihood objective similar to NMT, viz. (as a soft injection method).

$$p(Y|\hat{X}; \theta) = \prod_{t=1}^{T+1} p(y_t|y_{0:t-1}, x_{1:S}, C; \theta)$$

Model Architecture:

To achieve the UDAAN Tool's specification these are the methodologies implemented: They employed the standard Transformer-based architecture for NMT that uses self attention networks for both encoding and decoder.

1. Encoder:

- The encoder is a stack of N identical layers, each of which contains two sub-layers. Each layer consists of a multi-head self-attention (SelfAtt) and a feed-forward neural network (FFNN). For time step i in layer j , the hidden state $h_{i,j}$ is computed by employing self-attention over hidden states in the previous layer $j-1$.
- In the soft category, constraints are not forced into the predicted sentence but are used to guide the translation process.

- They define a segment as the sequence of constraints corresponding to the textual span between two <sep> symbols.
- In addition to this, they have appended input source token embedding with the learned embedding for the segment. This is illustrated in Figure below. Further, positional indices of the constraint tokens begin with a number that is larger than the maximum source sentence length. For a given token, its embedding is sum of three components, viz., token embedding, positional embedding and segment embedding.

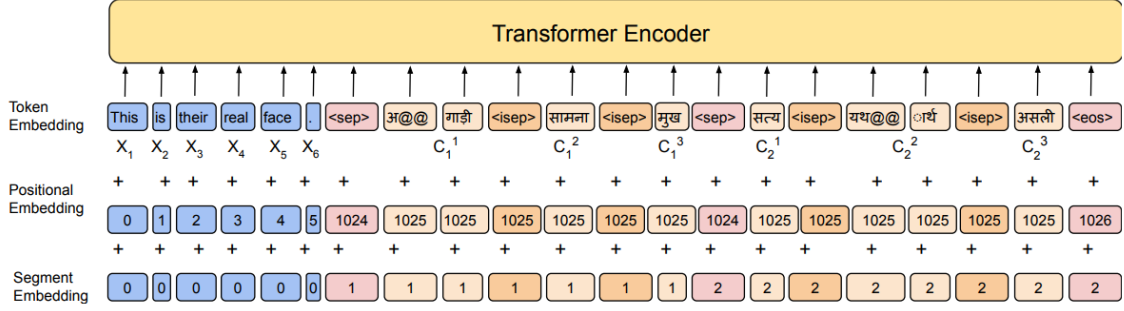


Figure 2: The embedding layer for the transformer-based encoder. Each source sentence is appended with the list of target constraints and separated by the special symbol '<sep>'. The target constraints are internally separated using the '<isep>' symbol to distinguish candidate constraints for the same source phrase.

2. Decoder:

- With a structure similar to the encoder, the decoder consists of a stack of N identical layers. In addition to two sub-layers, it also contains a cross attention (CrossAtt) sub-layer to capture information from the encoder. Cross attention is computed between the last layer of the encoder hidden state $h = h_{1,n}, h_{2,n} \dots h_{m,n}$ and the output of the self attention sub-layer.
- The probability for the token t is computed through a softmax over the target-side vocabulary, applied to the final decoder state s_N as:

$$P_t^{pred}(y_t | (y_{<t}, \hat{X})) = \text{softmax}(s_{t,N}, W)$$

Where, “W” is the learnable weight matrix,

“X” is the source Sentence,

“ $y_1, y_2, \dots y_T$ ” represent the target phrases.

- They also introduced a **pointer network** that adds logit values over the target-side constraints. Specifically, the token probability over the target vocabulary is the weighted sum of decoder probabilities from the predictive model P_t^{pred} and copy probabilities P_t^{copy} . This strengthens copying by identifying the target constraint (provided as an input) that needs to be copied. Copy probability for time step t is computed as the average multi-head attention weights of the last decoder layer. Intuitively, P_t^{copy} is the attention weight for the corresponding source position s .

- They also introduced a **pointer network** that adds logit values over the target-side constraints. Specifically, the token probability over the target vocabulary is the weighted sum of decoder probabilities from the predictive model P_t^{pred} and copy probabilities P_t^{copy} . This strengthens copying by identifying the target constraint (provided as an input) that needs to be copied. Copy probability for time step t is computed as the average multi-head attention weights of the last decoder layer. Intuitively, P_t^{copy} is the attention weight for the corresponding source position s .

$$P_t^{copy} = \text{avg}_{heads}(\text{CrossAtt}_{head_i}(\tilde{s}_{t,N}, h))$$

- We further add a **disambiguation network** to disambiguate between multiple inter-phrase constraints.
- This network learns to differentiate between multiple senses of a constraint based on the sentence context. We define disambiguation score for the j th inter-phrase component of the i th constraint, P_{ij}^{dis} as the dot product of weighted context vector and inter-phrase component (constraint component) contextual embedding normalised over all j inter-phrase constraints. This embedding of constraint -component combined with source context and cross attention can distinguish between multiple inter phrase constraints.

$$\begin{aligned} \text{score}_{ij}^{dis} &= c_t \cdot e_i^j = \sum_{s=1}^S \alpha_{t,s} h_s e_i^j \\ P_{ij}^{dis} &= \text{normalise}_j(\text{score}_{ij}^{dis}) \end{aligned}$$

Where, “ c_t ” is the weighted context vector,

“ e_i^j ” is the contextual embedding of interphrase constraints.

“ h_s ” is the encoder hidden state at position s of the last layer,

“ $\alpha_{t,s}$ ” is the averaged attention weight at the last decoder layer for source position s at decoding time step t . The final distribution over the target vocabulary is defined as the weighted sum of P_t^{pred} and the normalized probability sum of $P_t^{copy} + P_t^{dis}$

$$p(y_t | (y_{<t}, \hat{X})) = g_t P_t^{pred} + (1 - g_t)(P_t^{copy} + P_t^{dis}) \quad (5)$$

where $g_t \in [0, 1]$ is a learnable gate controlling the weightage of the two probabilities distributions. It decides whether to keep the provided user constraint in translation or let the model predict (as can be seen in Figure below). g_t is feedforwarded value of c_t (weighted context vector) and z_t (hidden state of the last layer of the decoder until timestep t)

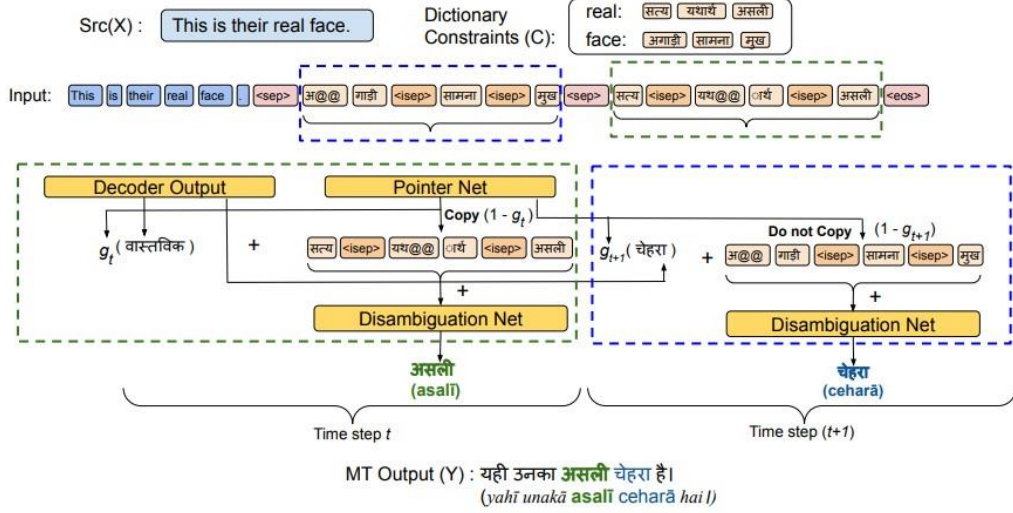


Figure 3: DICTDIS decoding mechanism. Gating value g_t regulates the final token probability which determines the decision of whether to disambiguate or to copy.

They also used a user-controllable parameter α which controls the aggressiveness of ingesting single-degree constraints. α is governed by the normalized cross attention on the constraints for each time step t . The final value of logits of constraints are increased by $\alpha \times \text{CrossAttt}$.

5.2 Cross-Language Span Prediction Technique for Bi-Directional Word Alignment using Multilingual BERT:

- The approach involves fine-tuning multilingual BERT embeddings to enhance alignment accuracy by aligning words with similar representations and reducing spurious alignments.
- In analogy to SQuAD(Stanford Question Answering Dataset), the word alignment system predicts translations of source words as spans in the target sentence, analogous to how question answering systems predict answers in a passage, they have modelled word alignment as a cross-language span prediction task akin to SQuAD-style question answering, where tokens in the source sentence(questions) predict spans in the target sentence(answers).
- Leveraging fine-tuned multilingual BERT on the Samanantar1 dataset, the model improves alignment accuracy by considering source token context and symmetrizing directional predictions.
- Alignments are converted into queries from tokens in one language to spans in the other, accommodating multi or null alignments. The question's language is treated as the source, and the context (and answer) language as the target,

maintaining consistency with SQuAD-style tasks. To handle null alignments, the SQuAD v2.0 format is adopted, explicitly addressing cases where there are no answer spans to the question in the given context.

- They defined the cross-language span prediction task as follows. Suppose we have a source sentence with $|X|$ characters $X = x_1, x_2, \dots, x_{|X|}$, and a target sentence with $|Y|$ characters $Y = y_1, y_2, \dots, y_{|Y|}$. Given source token $x_{i:j} = x_i \dots x_j$ that covers (i, j) in source sentence X , the task is to extract target span $y_{k:l} = y_k \dots y_l$ that covers (k, l) in target sentence Y .
- They solved this problem using multilingual BERT to this task, that added two independent output layers to the pretrained BERT to predict the start and end positions in the context.
- Suppose p_{start} and p_{end} are the probabilities that each position in the target sentence is the start and end positions of the answer span. They defined score $\omega_{ijkl}^{X \rightarrow Y}$ of target span $y_{k:l}$ given source span $x_{i:j}$ as the product of its start and end position probabilities and selected span (k, l) that maximizes $\omega_{ijkl}^{X \rightarrow Y}$ as the best answer span:

$$\omega_{ijkl}^{F \rightarrow E} = p_{start}(k|E, F, i, j) \cdot p_{end}(l|E, F, i, j) \quad (1)$$

$$(\hat{k}, \hat{l}) = \arg \max_{(k,l): 1 \leq k \leq l \leq M} \omega_{ijkl}^{F \rightarrow E} \quad (2)$$

- In the SQuAD model of BERT, first, the question and the context are concatenated to generate sequence “[CLS] question [SEP] context [SEP]” as input, where [CLS] and [SEP] are the classification and separator tokens, respectively. Then, the start and end positions are predicted as indexes to the sequence (if there are no answers, the start and end positions are the indexes to the [CLS] token).
- To make the span predictions more reliable, the authors has designed a simple heuristic to symmetrize the span predictions of two directions.
- As a symmetrisation (bi-directional alignment) method, called bidirectional average (bidi-avg), for an alignment we averaged the probabilities of the best spans for each token for each direction. A token is aligned if it is completely

included in the predicted span. We then extracted the alignments with the average probabilities that exceed a threshold.

- Let $x_{i:j}$ be a substring of sentence X that spans (i,j) , and let $y_{k:l}$ be a substring of sentence Y that spans (k,l) . Let $\omega_{ijkl}^{X \rightarrow Y}$ be the probability that token $x_{i:j}$ predicts span $y_{k:l}$, and let $\omega_{ijkl}^{Y \rightarrow X}$ be the probability that token $y_{k:l}$ predicts span $x_{i:j}$. Let ω_{ijkl} be the probability of alignment a_{ijkl} where token $x_{i:j}$ is aligned to token $y_{k:l}$. We define ω_{ijkl} as the average of probability $\omega_{X \rightarrow Y}^{ij \hat{k} \hat{l}}$ of the best predicted span $y_{\hat{k}:\hat{l}}$ from $x_{i:j}$ and probability $\omega_{Y \rightarrow X}^{\hat{i}jkl}$ of the best predicted span $x_{\hat{i}:\hat{j}}$ from $y_{k:l}$.

$$\omega_{ijkl} = 1/2(I_{\hat{k} \leq k \leq l \leq \hat{l}}(\omega_{ij\hat{k}\hat{l}}^{X \rightarrow Y}) + I_{\hat{i} \leq i \leq j \leq \hat{j}}(\omega_{\hat{i}jkl}^{Y \rightarrow X})) \quad (3)$$

where $I_A(x)$ is an indicator function. $I_A(x)$ returns x if A is true and 0 otherwise.

- The authors considered a threshold of 0.4 for ω_{ijkl} to consider $x_{i:j}$ and $y_{k:l}$ as aligned. Also, the original implementation of the SQuAD model only outputs an answer string, but the authors extended it to output the answer's start and end positions.

5.3 Lexicon Suggestions using SPEAR library:

SPEAR is an open-source Python library designed for data programming with semi-supervised learning. It facilitates the creation of training data by programmatically labelling unlabelled data using weak supervision methods.

- This labelling of unlabelled data done using “Labelling functions.” Labelling functions (LFs) are rules or heuristics defined by users to programmatically assign labels to unlabelled data in a semi-supervised learning setting. These functions encode domain knowledge or patterns observed in the data to automatically generate labels, thereby assisting in the creation of labelled datasets for training machine learning models.
- The SPEAR package consists of three components (and they are applied in the same order): (i) Designing LFs, (ii) applying LFs, and (iii) applying a label aggregator (LA).

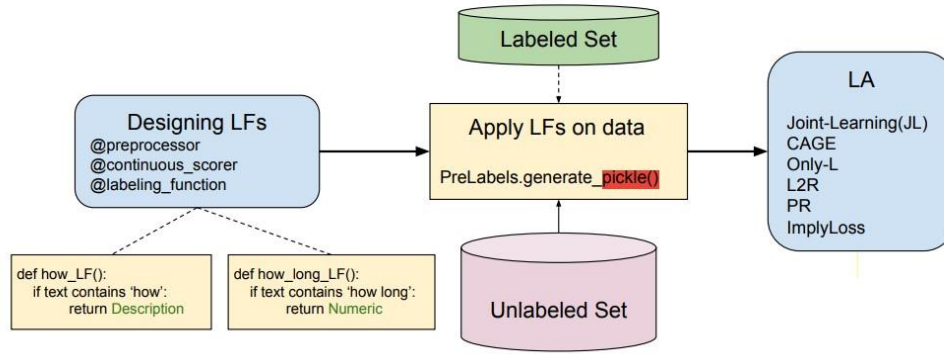


Figure 1: Flow of the SPEAR library.

- The LF can express pattern matching rules in the form of heuristics, distant supervision by using external knowledge bases and other data resources to label datapoints.
- SPEAR uses a `@labeling_function()` decorator to define a labeling function. Each LF, when applied on an instance, can either return a class label or not return anything, i.e. abstain.
- SPEAR is employed in project UDAAN4 for reducing post editing efforts. UDAAN (Maheshwari et al., 2022b) is a post-editing workbench to translate content in native languages.
- Based on the post editor's patterns of changes to the target language document, candidate labeling functions are generated (based on a combination of heuristics and linguistic patterns) by the UDAAN workbench (Figure 4 for examples of LFs).
- Based on these LFs, SPEAR gets invoked on a combination of the edited (i.e., labelled) data and the not yet edited (i.e., unlabelled) data to present consolidated edits to the post-editor. This use case has been presented in the flow chart in Figure 4.

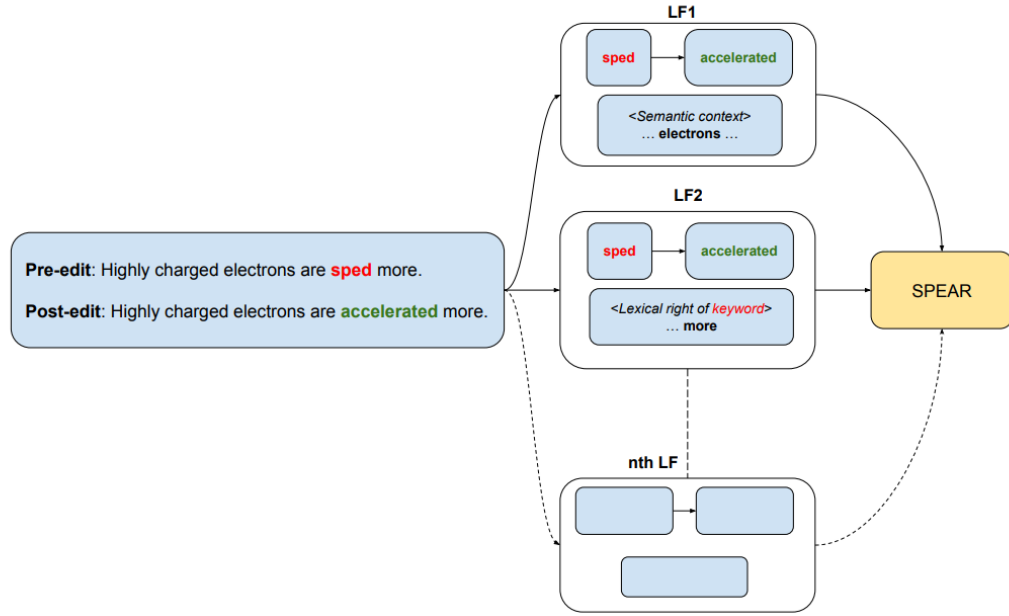


Figure 4: Mutiple LFs generated from post-editor edits based on semantic and lexical features while editing science (domain-specific) document in English.

5.4 Tesseract4:

SPEAR is an open-source Python library designed for data programming with semi-supervised learning. It facilitates the creation of training data by programmatically labelling unlabeled data using weak supervision methods.

- Tesseract4 is an open-source Optical Character Recognition (OCR) engine developed by Google. It employs a combination of image processing, machine learning, and natural language processing techniques for converting images containing printed or handwritten text into machine-readable text.
- In Tesseract4, LSTM networks are integrated into the recognition pipeline to improve the accuracy of character recognition. LSTM networks are trained on large datasets of labelled text images to learn patterns and relationships between characters. During inference, these trained LSTM models are used to predict the most likely characters for each image region, considering contextual information and long-range dependencies within the text.
- The Input files like pdfs, docs given to this Tesseract will first gets convert into image and then, they will be pre-processed for OCR.
- The authors have fine-tuned the model based on preliminary data collected from the tool on Devanagari font and claimed that their fine-tuned model performs significantly better than the default version provided by Tesseract.

CHAPTER 6

SYSTEM IMPLEMENTATION

This chapter deals with the implementation of UDAAN tool starting from the raw pdf file to complete domain aware translation. This chapter will describe the literal pipeline being followed by the project to perform translation task.

The tool can be found on [UDAAN download](#) website. The website is seldom down therefore the zip of the tool can be found attached with this report.

6.1 Installation Requirements

The UDAAN tool is available for Windows and Ubuntu which could be selected from the above-mentioned link.

6.2 Workflow pipeline

The first step in the workflow is account creation [English to Hindi](#) for Hindi to English or vice versa, or [Indic-Indic](#) which includes translation from 12 Indic languages to 16 Indic languages including English.

Instructions:

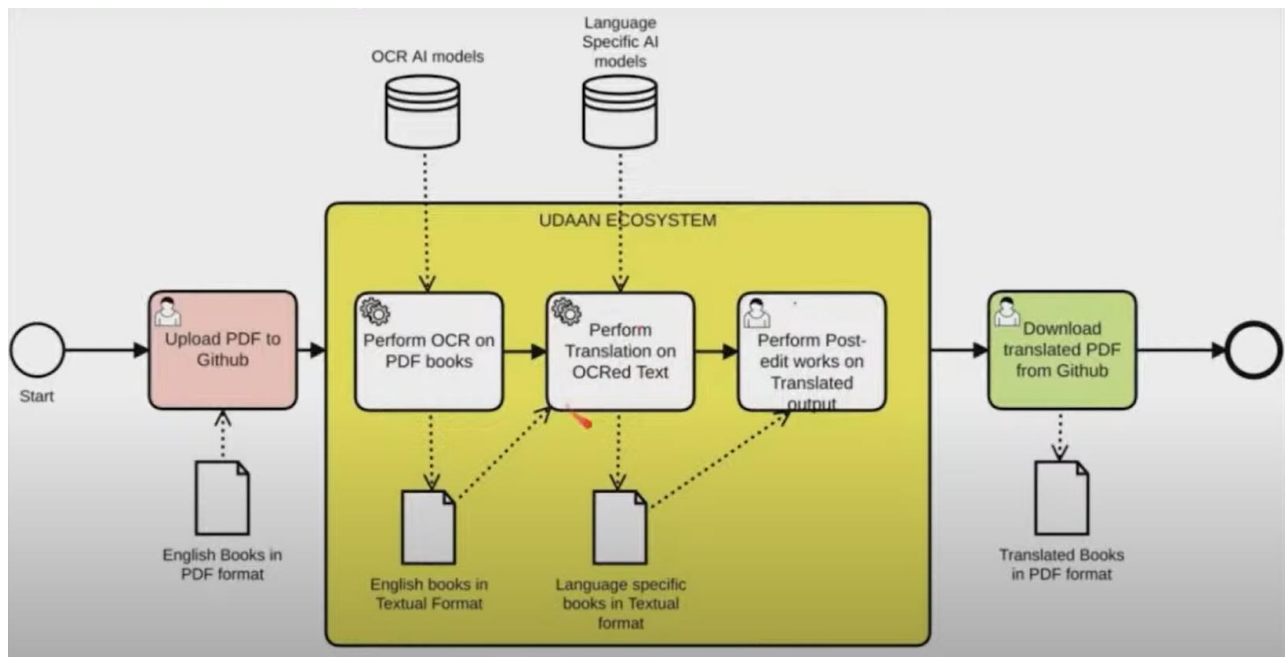
1. Uploading PDF, DOC, EXCEL or PPT file size should be less than **1024 MB**.
2. Document name should be greater than 5 characters.
3. First upload the PDF, DOC, EXCEL or PPT file, then enter other fields for OCR, Translation.
4. Number of Auto Select Dictionaries should be less than or equal to 64.
5. Your Document will be added to the queue once you click the convert button, and you will receive an email once the OCR or Translation is Finished.
6. Large PDFs, DOCs, EXCELS or PPTs will take hours to complete the OCR or Translation, so please be Patient!

The web interface of UDAAN tool is provided below which is self-explanatory.

The screenshot displays the UDAAN tool's web interface. At the top, there is a file upload section with the text "Drop your file here or Click to browse". Below this is a dashed box containing a "Choose File" button and the text "No file chosen". A purple "Upload" button is positioned below the file upload area. The interface includes several dropdown menus: "Select Action" (set to "OCR & Translation"), "Document Name" (with a placeholder "Enter Document Name"), "Select Source Language" (set to "English"), and "Select Target Language" (set to "Hindi"). There is a "Select Dictionaries" section with a search bar and a list of checkboxes for various fields: None, Administrative, Agriculture, Agricultural Terminology, Applied Geology Terms Glossary, and Biology Learners Glossary. To the right of the dictionary list are three toggle switches: "Generate output as docx", "Is Document Handwritten?", and "Auto Select Dictionaries". Below these is a "Number of Dictionaries You Want To Auto Select" slider set to 5. At the bottom right, there is a toggle switch for "Sentence with the English translation enclosed in brackets" with an example: "Eg: education - शिक्षा (education)". A large orange "Convert" button is located at the bottom center of the interface.

The above snapshots contain a file input where the user can input a pdf in any of the 16 given languages in “**Select Source Language**” to get translation in any of the 16 languages present in “**Select Target Language**”. There is an option to select multiple dictionaries from the list “**Select Dictionaries**” that range from Administration to Zoology. There are a total of 65 dictionaries in Indic-Indic translation. The user can also select if the uploaded document is handwritten and a feature to auto select dictionaries. For the user convenience, it also provides a feature to keep the word with the translation for better understanding of translation.

The background operations of UDAAN as given by the developers is given below:



After the submission has been made by uploading document and selecting all the required information, the portal says to wait for a day to get the translated document in the mail within 24hours or the document could be found on **Preview/Download**.

Home Udaan Website QuickLink Translation Preview/Download Add collaborators Anuja								
Preview/Download Table								
Sr. No	File Name	Source Language	Target Language	Document Name	Dictionary Used	Action Used	Preview	Download
1	ArchP5.pdf	en	hi	AICTE1Testing2	administrative	ocr_translation	Preview	Download
2	rfp-for-selection-of-merchant-banker.pdf	en	hi	testingbook.trpf	administrative,rti	ocr_translation	Preview	Download
3	sms-registration-form6.pdf	en	hi	testingsmsudaan	administrative,rti	ocr_translation	Preview	Download
4	sms-registration-form6.pdf	en	hi	sms-registration-form656	administrative	ocr_translation	Preview	Download

NOTE - The above image has been taken from the official Installation Guide since we did not get the translated text from the machine.

After downloading the translated file from the website, the user can open MT file on UDAAN tool for post editing which provides 3 roles to the user namely-

1. **Corrector** - who edits a document
2. **Verifier** - who validate the output from Correctors
3. **Proofreader** - who validate the final translated document.

In the corrector mode, the user can perform changes on MT file to further improve the domain aware translation.

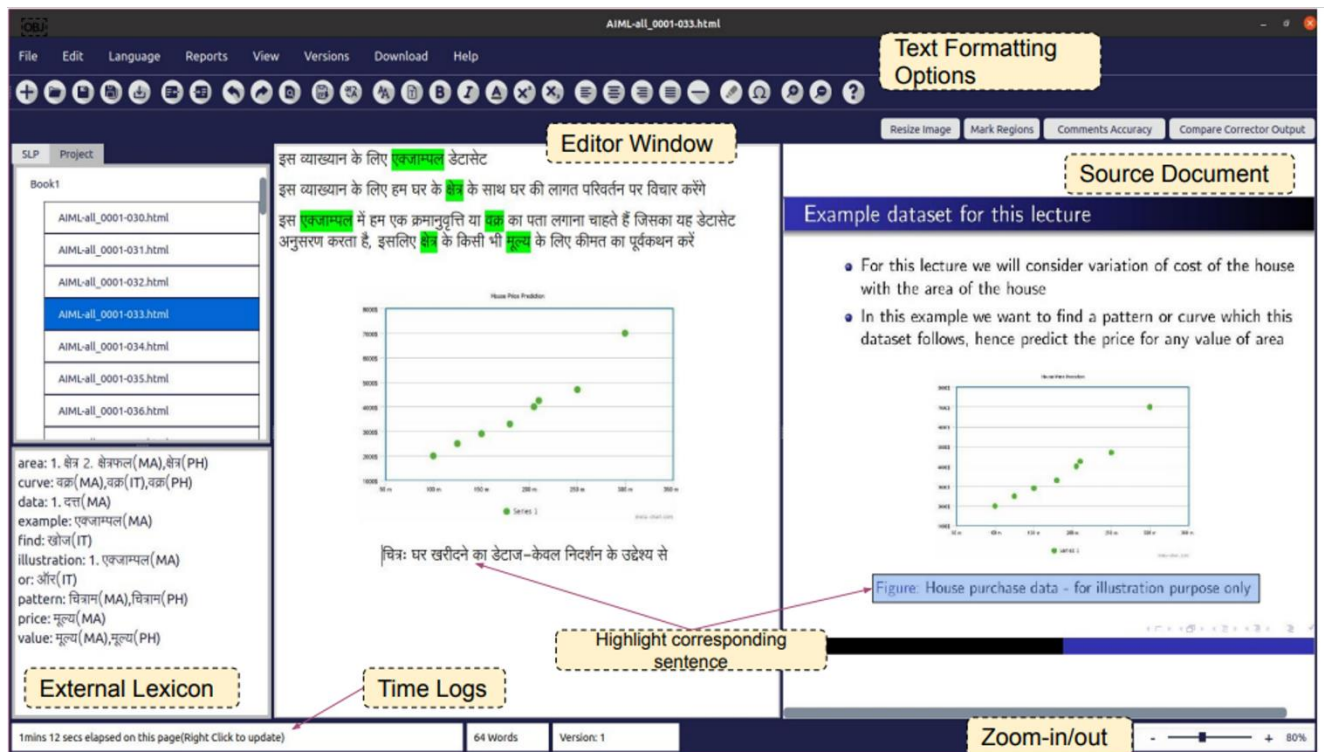
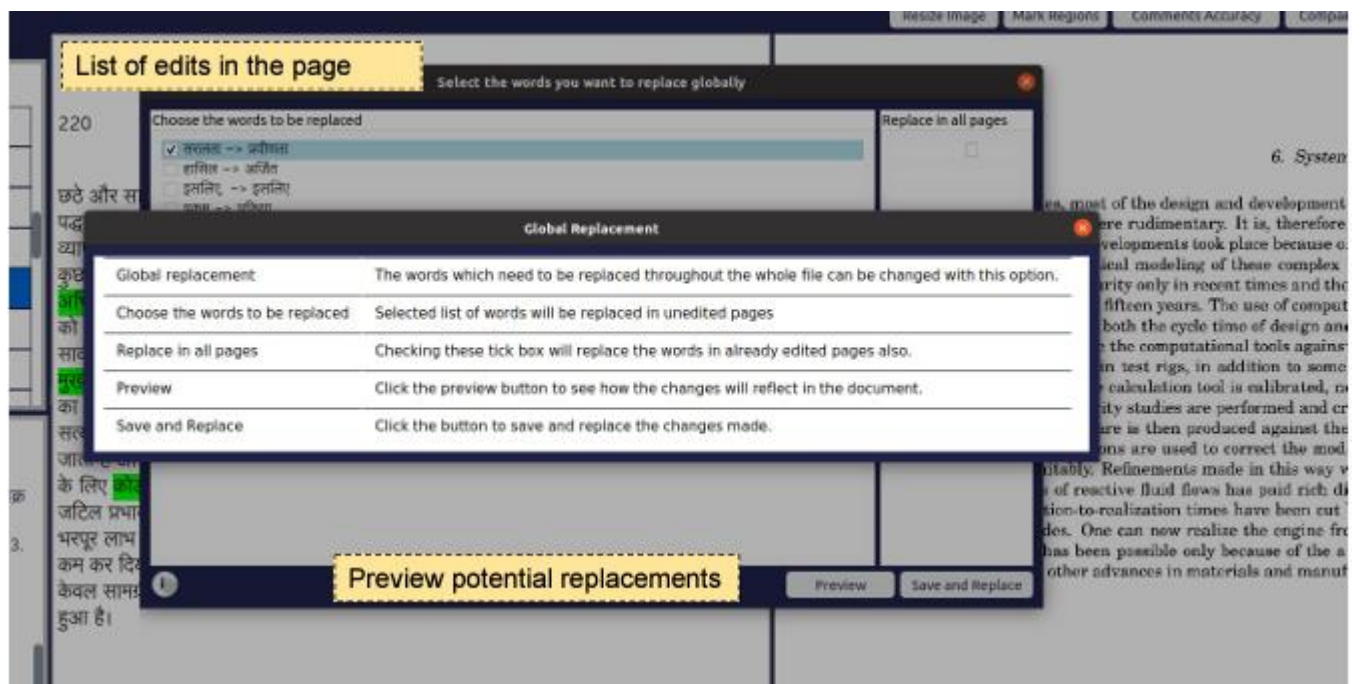


Figure 1: Screenshot of the UDAAN tool with highlights about its key features.

NOTE – The image has been taken from the official research paper of UDAAN tool.

The tool also provides a means to perform global or local word replacement based on the user past activities.



The user can also keep track of edits based on global replacements or technical dictionary suggestions implemented by highlighting the words using different colors. Yellow refers to globally replaced words and green highlight refers to dictionary word replacement.

नेशनल गैलरी **आणि** मॉडर्न आर्ट, नई दिल्ली

अब इसे **राष्ट्रीय** **आधुनिक** कला संग्रहालय, नई दिल्ली में प्रदर्शित करवाया गया है।

एक कलाकार, देवयानी कृष्णा ने एचिंग प्रिंट का उपयोग करके लिंगम नंबर 1,1990 नामक **एक कलात्मक कार्य** बनाया। इसे अब **राष्ट्रीय** **आधुनिक** कला दीर्घा, नई दिल्ली में प्रदर्शित करवाया

CHAPTER 7

ANALYSIS

- **Code mixing:** We tested the tool with mixed code to check the efficiency and accuracy of the model but to our disappointment the tool did not perform to our expectations and gave wrong results. The tool gave an error and could not complete the translation.
 - We suggest using code switching detection and augmenting code-mixed datasets.
- **No auto recognition of source language:** Upon intense analysis of the tool we found that one needs to explicitly mention the language of the input text. This works well for most cases except when the user wishes to convert the file to a target language unaware of source language.
 - We suggest using code switching detection and augmenting code-mixed datasets. Langdetect, TextBlob etc. Can also be used to automatically detect languages.
- **Low resource language:** The UDAAN model has been trained on “**Samanantar1**” dataset which had 12.4M which was preprocessed by the developers and reduced to 8.46M. Since it covers only 16 languages, we believe that the amount of data used to train the data is not sufficient and hence does not properly cover all the languages. Had the data been larger and diverse, the model probably might perform well.
 - We suggest using Neural Machine Translation of Low Resource and Similar Languages with Backtranslation. Semi supervised NMT can also be used to overcome with this shortcoming.
 - In our talk with Ajay Ravindran (MTECH, IITB), he mentioned their current work on using “**Samanantar2**” to enhance knowledge base of the model.
- **Transliterated text:** We tried to compare the tool with GOOGLE Translate tool to check the performance of the tool. We were satisfied to know that UDAAN works well transliterated text where Google translate failed. This implies that UDAAN has been well trained to handle unambiguous and transliterated text.
- **Common sense adaptability:** Upon intense analysis of the tool on various examples and word permutations, we figured that the tool performs translation using the domain knowledge but does not apply any common sense to the translation. We understand that this tool is built to provide domain aware translation but ignoring the sense could drastically change the meaning of the sentence. Eg. (पेन में बॉक्स रखा है) was translated to the box is in the pen, which can surely not be the case.
 - Semantic networks could be used here which could assist the tool keep the sense in translation.

- **Data privacy:** As mentioned above about the workflow of the tool. The file needs to be uploaded on the server for domain aware MT which makes it unreliable for private data. Even if IITB maintains complete privacy and security of data, passing private data to other server is still risky.
 - If the model weights and tokenizers would have been easily available somewhere, huggingface for instance, we could perform the same task on critical data as well.

CONCLUSION

We analyzed the UDAAN project, which is a post-editing tool for translation which uses OCR. The pipeline of UDAAN consists of producing lexically constrained NMT, which is further used for post-editing using the tool. From our analysis, we identified that UDAAN has provided several features that significantly improve the quality of translation like lexically constrained translation, bidirectional word alignment using advanced techniques as discussed. Compared to translation from scratch and post-editing over MT, studies show that UDAAN tool achieved 3-3.5 times increase in speed. Finally, the applicability of UDAAN tool was well evaluated. From discussions with the members of UDAAN developers, they said that they are working to incorporate a larger dataset of “**SAMANANTAR**” dataset and add more languages. They are currently developing a web interface that would allow users to work in online mode. They were also planning to make their replacement module robust by including context-sensitive and morphologically aware replacement techniques for suggesting replacements. These additional features will further help reduce the post-editing effort. Also, we [hope that they will consider the limits](#).

BIBLIOGRAPHY

- [UDAAN - Machine Learning based Post-Editing tool for Document Translation](#)
- [SPEAR : Semi-supervised Data Programming in Python](#)
- [DICTDIS: Dictionary Constrained Disambiguation for Improved NMT](#)
- [A supervised word alignment method based on cross-language span prediction using multilingual BERT](#)
- [Samanantar: The Largest Publicly Available Parallel Corpora Collection for 11 Indic Languages](#)
- <https://cstt.education.gov.in/sites/default/files/mou-iit-cstt.pdf>
- Various other research papers from ResearchGate and IEEE
- Websites: Google, TowardsDataScience, AnalyticsVidya, ChatGPT